Wizardry — fourth wave: how any of this gets built

Written 12 November 2026. Waves one to three described what should exist; this one describes how it comes into existence, because there is a gap nobody has looked at yet.

1. The uncomfortable fact

The running instance is **upstream images**. Every one of them is `ghcr.io/fluxerapp/*` pulled by digest and left untagged — app-proxy 874 MB, api 663 MB, gateway 636 MB, and nine more upstream images across twenty running services. Nothing on that machine is built from this fork.

So the twelve rebrand files in the repository — `ProductConstants.ts`, `DesktopIdentity.ts`, `instance_branding.rs`, the billing strings, the desktop config — are **dormant**. What is live is a *skin*: the branding row, the custom CSS, the artwork. That is enough to look like Wizardry. It is not enough to *be* it, and it means every code fix planned in wave three (the manifest colour, the boot watchdog, the offline page) currently has no route to the server.

Fact	Value	How known
Images in use	`ghcr.io/fluxerapp/*`, all `<none>` (digest-pinned)	`docker images` on `wizardry-01`
Fork files changed for the rebrand	12	`git log` / `REBRAND.md`
Deployed from this fork	**0**	no image in the local registry
Build workflow	`.github/workflows/build-app-proxy-self-hosted.yaml`, 5 jobs, `runs-on: ubuntu-24.04`	workflow file
Credentials that workflow needs	`actions/create-github-app-token` — a GitHub App private key this fork does not have	workflow, lines 61+
Last CI run	queued, never picked up	earlier session
Local toolchain here	Docker 26.1.5, node 26.5.1, **no pnpm, no cargo**	`--version`
Build dependencies	all inside the Dockerfile: `node:26-trixie-slim`, `pnpm@11.27.0`, `dtolnay/rust-toolchain`	`fluxer_app_proxy/Dockerfile`
Server disk free	61 GB of 75 GB	`df -h`

Two conclusions follow, and they are the reason this document exists. **CI in this fork cannot run** — it targets GitHub-hosted runners and asks for an app token that was never issued, which is why the badge says queued and nothing happens. And **the build has to happen where Docker is**, which is either this machine or the server, not GitHub.

2. Four items

4.1 A build that runs at all

First, because everything else is fiction until an image exists.

Build **one** image — the app-proxy, because it carries two of the three code fixes — on the server, from a clean checkout of the fork at a known commit.

- Rules: build from a tagged commit, never from a dirty tree; tag the result with that commit (`wizardry/app-proxy:<sha8>`), never `latest`; if the build fails, the failure is the finding and the wave stops there rather than being worked around.
- Pass: a build completes and prints its duration; the image is under 1.2 GB; `docker run` answers its health endpoint; `docker image inspect` shows the commit in `org.opencontainers.image.revision` (set `BUILD_VERSION` if it does not).

4.2 A deploy with a way back

Second, because a build that replaces a working proxy is a risk, not progress.

- Rules: the upstream image is preserved by **digest** before anything is replaced, so the revert command names a digest and not a guess; the swap touches one service; the result is proven by the gate (wave 1, 1.3), not by "it started".
- Pass: the upstream digest is recorded; the service runs the local image; the gate passes 8/8; a revert is executed once, deliberately, and takes under two minutes.

4.3 The two fixes that need the build

Third, because this is what the build is for.

The manifest `background_color`, and the boot watchdog plus the worker's offline fallback (both specified in wave 3 as 3.1 and 3.6).

- Rules: one build carries both; the watchdog is tested by throttling to offline in a real browser, not by reading the code; the offline page is self-contained — no fonts, no images, no network — so it works precisely when nothing else does.
- Pass: `manifest.json` returns `background_color: #18271c`; with the network set to offline, the branded message and Retry control appear within eight seconds; the worker's cache name carries the version and the previous generation is deleted on activate.

4.4 Settle the fork's dormant files

Last, because it is about honesty rather than features.

Twelve files were changed for the rebrand and none of them are deployed. Each one has to end up in one of two states: **deployed and verified**, or **reverted with a note saying why**. A third state — present in the repository, absent from production — is how a fork becomes a costume.

- Rules: a file is only "deployed" if an image built from it is running; `REBRAND.md` records which image carries which change; anything that cannot ship is struck from the document in the same commit that reverts it.
- Pass: `REBRAND.md` lists every changed file with the image that carries it; nothing is listed as shipped unless `docker image inspect` on the running service agrees.

4.5 The CI decision

Alongside 4.4, because a broken badge is worse than no badge.

Either put a self-hosted runner on `wizardry-01` so the workflows can run where Docker already is, or delete the workflows and say in the README that builds are local. Leaving five jobs that queue forever is the only option that is definitely wrong.

- Rules: a workflow that cannot authenticate must fail loudly, not wait; if a runner is added, it is scoped to this repository and nothing else.
- Pass: either a workflow run completes green on our own runner, or `.github/workflows/` is empty and the READMEs explain why.

2a. What actually happened (12 November, same day)

The build works. `wizardry/app-proxy:<sha>`, 874 MB, about twelve minutes cold and six warm, SPA plus Rust proxy, on `wizardry-01`. The falsifier in §6 did not trigger.

Three things were learned by doing it, and all three are load-bearing:

1. **`FLUXER_SELF_HOSTED=true` is not optional.** The first image built without it and the proxy came up serving **upstream defaults** — `/manifest.json` said `Fluxer`, `#4641D9`, `#2b2d31`. The branding merge only compiles into the self-hosted variant. The workflow already passes that arg (`.github/workflows/build-app-proxy-self-hosted.yaml` line 180); a hand-run build has to pass it too.
2. **Deploy is `custom.compose.yml` plus `rexctl restart`.** Compose directly is refused — rexctl guards the stack with `FLUXER_SETTINGS_LOADED` and tells you to use rexctl or `rexctl eject`. The supported override is a service block in `custom.compose.yml`, which rexctl keeps across upgrades.
3. **The revert is restoring one file.** `cp custom.compose.yml.orig custom.compose.yml` and a restart put the container back on the upstream digest `sha256:779632ee…` — executed deliberately, twice, in under a minute each.

2b. Correction

Wave one said a restore is manual because `rexctl backup` offers only keep, list, location and verify. That was wrong: **`rexctl restore <file>` exists as a top-level command** — I read `backup --help` and stopped. The drill run here (extract, `sha256sum -c`, `pg_restore -l`) is still worth having as a smoke test, but the real path back is one rexctl command.

2c. The message index, re-measured

Wave two called the empty `messages` index a defect. The measurement was right; the inference was not. What is actually true:

- Every one of the nine channels has `indexed_at = NULL`. No channel has ever been indexed.
- The `messages` index holds 0 documents and was last written at 14:01:07 — the second it was created.
- Messages sent at 15:43, 22:16 and 22:26 were not indexed. The API logged **no** attempt and no error: `MessageSearchService.indexMessage` returns silently when `getMessageSearchService()` is null, and nothing else logs on that path.

- Indexing is designed as a **per-channel job** (`indexChannelMessages`), queued by `queueIndexingChannels` from `channelNeedsReindexing(channel.indexedAt)` — which is true for all nine channels — i.e. triggered by a search, not by sending a message.

So the index is *cold*, not necessarily broken, and the two are told apart by one action: search for a word that is in a message. If results appear and the index fills, the design is lazy and correct. If they appear but the next message does not index, send-time indexing is the defect. Until one search has run, "0 documents" measures the instance's age, not its health.

3. Order

1. **Restore drill** (wave 1, 1.2) — before any deploy, not after. A backup that has never been restored is not a safety net, and this wave is the first that can break the site.
2. **4.1 build the proxy** — the whole wave hinges on whether that succeeds.
3. **4.2 deploy with a recorded way back.**
4. **4.3 the manifest and the watchdog** — the reason the build exists.
5. **4.4 dormant files** — decide, file by file.
6. **4.5 CI** — last, because its answer depends on whether the build is a one-off or a routine.

4. Considered and rejected

- **Building all twelve upstream images from the fork.** Only three carry our changes, and each build is minutes of CPU and hundreds of megabytes. Build what we touch.
- **Running the monorepo's full test suite before the first build.** It is a large suite against Cassandra and Meilisearch; the honest gate for this wave is the eight-check render gate, which fails on the things a user sees.
- **Replacing `rexctl` with hand-written compose.** `rexctl` owns boot persistence, upgrades and doctor; hand-rolling that is how a self-hosted instance becomes a hobby project with no upgrade path.
- **GitHub-hosted CI.** It is what the workflows assume and what does not work here.
- **Vendoring crates for an offline build.** The server has network; vendoring would be solving a problem we do not have.
- **A brand-new `wizardry` image namespace and registry.** A local tag with the commit in it is enough to be honest about what is running; a registry is infrastructure for a second machine we do not have.

5. Correction

Wave three called the manifest fix "one field", which was true of the patch and misleading about the delivery: that field lives in Rust inside an image that has never been built from this fork, so it costs a Docker build, a deploy and a revert path before anyone sees a different splash colour. The same is true of the watchdog. "Small patch" and "small change" are different claims, and this document exists because I conflated them.

6. What would falsify this wave

If the first app-proxy build fails in a way that needs upstream's own toolchain (a missing private dependency, a build arg only their CI provides), then the fork cannot carry code fixes at all, and the honest plan collapses to: run upstream images, keep the skin, and report the two defects upstream instead of patching them. That is a legitimate outcome; it should be written down rather than fought.

7. When to stop planning

This is the fourth document and the last one. It describes work whose first step is a command, not a paragraph. If a fifth is ever written before `wizardry/app-proxy:<sha>` exists, the planning has become the project.

Change log

- 12 Nov 2026 — first issue. Written after noticing that no image on the server came from this repository, and that the fork's CI cannot authenticate.